

Reporte de Proyecto: Método Monte Carlo Iterativo

Jesús Galaviz Tapia

Introducción

En este proyecto se implementó un método Monte Carlo iterativo para aproximar la solución de una **ecuación diferencial ordinaria (ODE) de segundo orden no lineal** con condiciones de frontera de Dirichlet:

$$\frac{d^2y}{dx^2} = F\left(x, y(x), \frac{dy}{dx}\right) \quad x \in (a, b), \quad y(a) = Y_a, \quad y(b) = Y_b.$$

El algoritmo está basado en el método propuesto en [1]. Este método consiste en simular **random walks** sobre una malla discreta que representa el intervalo $[a, b]$, con el objetivo de obtener una aproximación Y de la solución de la ODE en el i -ésimo punto de la malla; es decir, se obtienen valores $Y(x_i) = Y_i$. Inicialmente se parte de una función $f(x)$ (que satisface las condiciones de frontera) como primera aproximación. Después, para cada punto interior de la malla, se estima un nuevo valor promediando muchos random walks que inician en ese punto y se detienen al llegar a una frontera (condiciones absorbentes). Al terminar este procedimiento se obtiene el conjunto de valores Y_i . Este conjunto se utiliza como nueva $f(x)$ en la siguiente iteración para generar una nueva aproximación. El proceso se repite varias veces, mejorando la exactitud de los resultados.

La implementación se realizó en **Python** empleando las librerías **NumPy** y **Numba**.

- **Numba** se utilizó para compilar el código a bajo nivel (backend en C), reduciendo significativamente el tiempo de ejecución comparado con el intérprete de Python. Además, permitió la paralelización del algoritmo mediante el decorador `@jit(parallel = True)`. Numba paraleliza códigos usando threads en el **CPU**.
- **NumPy** se usó para el manejo eficiente de vectores (arrays) y operaciones vectorizadas, las cuales son compatibles con **Numba**. Para los random walks se usó el generador de números enteros con distribución uniforme `np.random.randint()`. Este generador está incluido en **NumPy** y se basa en el algoritmo **Mersenne Twister** (MT19937), cuyo periodo de recurrencia es $2^{19937} - 1$.

El programa genera un archivo de texto en formato **(x,y)** que especifica la coordenada x y y , respectivamente, de la solución aproximada.

Parámetros utilizados

Para correr el programa (serial o paralelo) sólo se necesita especificar los siguientes parámetros que están al inicio del código, así como las funciones $F(x, Y, Y')$ y $f(x)$ (que es la primer aproximación a $y(x)$ y por default en el programa es una línea recta).

- N : número de puntos en la malla.
- a, b : fronteras del dominio.
- Y_a, Y_b : valores de frontera.
- K : número de caminantes aleatorios por punto de la malla.
- `set_num_threads( $n$ )`: n es el número de hilos usados en la ejecución paralela.
- **NAME**: nombre del archivo de salida donde se guardan los resultados.

Metodología

El intervalo $[a, b]$ se discretiza en una malla uniforme de N puntos:

$$x_i = a + i h, \quad \text{con} \quad h = \frac{b - a}{N}.$$

Para cada punto interior x_i , se generan K caminantes aleatorios. Cada caminante realiza pasos a la izquierda o derecha con igual probabilidad. En cada paso se evalúa el término $F(x, Y, Y')$ correspondiente a la ODE, contribuyendo a la nueva aproximación en ese punto, como se indica:

el esquema de aproximación en el punto i -ésimo de la malla es:

$$Y^{(s+1)}(x_i) = \frac{1}{K} \sum_{j=1}^K \left[Y^{(s)}(x_{\text{end}}^{(i,j)}) - \frac{h^2}{2} \sum_{l=0}^{m_{i,j}-1} F\left(P_l^{(i,j)}, Y^{(s)}(P_l^{(i,j)}), (Y^{(s)})'(P_l^{(i,j)})\right) \right]$$

donde:

- $P_l^{(i,j)}$ es el l -ésimo punto visitado por el j -ésimo caminante, iniciando en x_i .
- $x_{\text{end}}^{(i,j)} = P_{m_{i,j}}^{(i,j)}$ es el punto frontera alcanzado por el caminante j ;
- $m_{i,j}$ es el número de pasos realizados hasta alcanzar una frontera.
- $Y^{(s)}(x_i)$ es la aproximación obtenida en la iteración s .
- $Y^{(s+1)}(x_i)$ es la nueva aproximación en el i -ésimo punto.

De esta manera $Y_i^{(0)} = f(x_i)$ y $P_0^{(i,j)} = x_i$.

Este proceso se repite q veces, obteniendo en cada iteración una aproximación más precisa. Su implementación se realizó como sigue:

- La función $Y_C_i()$ realiza K *random walks* iniciando en el punto x_i . A lo largo del conjunto de puntos $\{P_l^{(i,j)}\}$ $l = 0, 1, 2, \dots, m_{i,j} - 1$ visitados por el caminante j , se acumula

$$U_{i,j} = \sum_{l=0}^{m_{i,j}-1} F\left(P_l^{(i,j)}, Y^{(s)}(P_l^{(i,j)}), (Y^{(s)})'(P_l^{(i,j)})\right)$$

Cuando el caminante haya alcanzado algún punto frontera se obtiene el término

$$V_{i,j} = Y^{(s)}(x_{\text{end}}^{(i,j)}) - \frac{h^2}{2} U_{i,j}$$

Después $V_{i,j}$ es sumado a un acumulador local. Cuando todos los K random walks se han realizado, se habrá calculado

$$\sum_{j=1}^K V_{i,j}.$$

La función entonces regresa la cantidad anterior dividida por K , obteniendo así el promedio

$$Y^{(s+1)}(x_i) = \frac{1}{K} \sum_{j=1}^K V_{i,j}.$$

que representa la nueva aproximación de Y en el punto x_i .

Para el algoritmo paralelizado, esta función es la que se ejecuta en paralelo; es decir, se generan n random walks simultáneamente (según el número de threads que se hayan especificado). No hay *race conditions* porque el valor $U_{i,j}$ solo se acumula en una variable local de la función y nunca es compartido entre threads.

- La función $Y_C()$ calcula la derivada numérica de la aproximación $Y^{(s)}$, después aplica la función $Y_C_i()$, de manera serial, a todos los puntos interiores de la malla, generando la nueva aproximación $Y^{(s+1)}$.
- Finalmente, la función $MCS()$ itera este proceso q veces de manera serial, generando una secuencia de aproximaciones:

$$Y^{(1)}, Y^{(2)}, \dots, Y^{(q)},$$

de modo que la última de ellas, $Y^{(q)}$, representa la solución numérica aproximada de la ecuación diferencial.

Configuración de hardware

La implementación se realizó en una **MacBook Air M1 (2020)** con las siguientes características:

- CPU de 8 núcleos, arquitectura ARMv8-A de 64 bits.
- 4 de los núcleos a 3.2 GHz y los otros 4 a 2.1 GHz.

- Con cachés de 12 MB y 4 MB, respectivamente.
- Ancho de banda de memoria de 68.25 GB/s.

Resultados obtenidos

Para la ecuación diferencial:

$$\frac{d^2y}{dx^2} = F\left(x, y(x), \frac{dy}{dx}\right) = \pi^2 \sin(\pi x) + e^{x+\sin(\pi x)} - e^{-y} \quad x \in (0, 1), \quad y(0) = 0, \quad y(1) = -1$$

la solución exacta es $y(x) = -x - \sin(\pi x)$. Para aproximar esta solución se usaron los parámetros $N = 100$ y $K = 10^5$ (además de las condiciones de frontera).

Las figuras siguientes muestran los resultados obtenidos por este trabajo y los obtenidos en [1], para la ecuación diferencial mencionada. Los tiempos de cómputo fueron:

- $q = 1$: 349.333 s para el algoritmo serial, 97.651 s para el algoritmo paralelo.
- $q = 10$: 3478.632 s para el algoritmo serial, 955.149 s para el algoritmo paralelo.

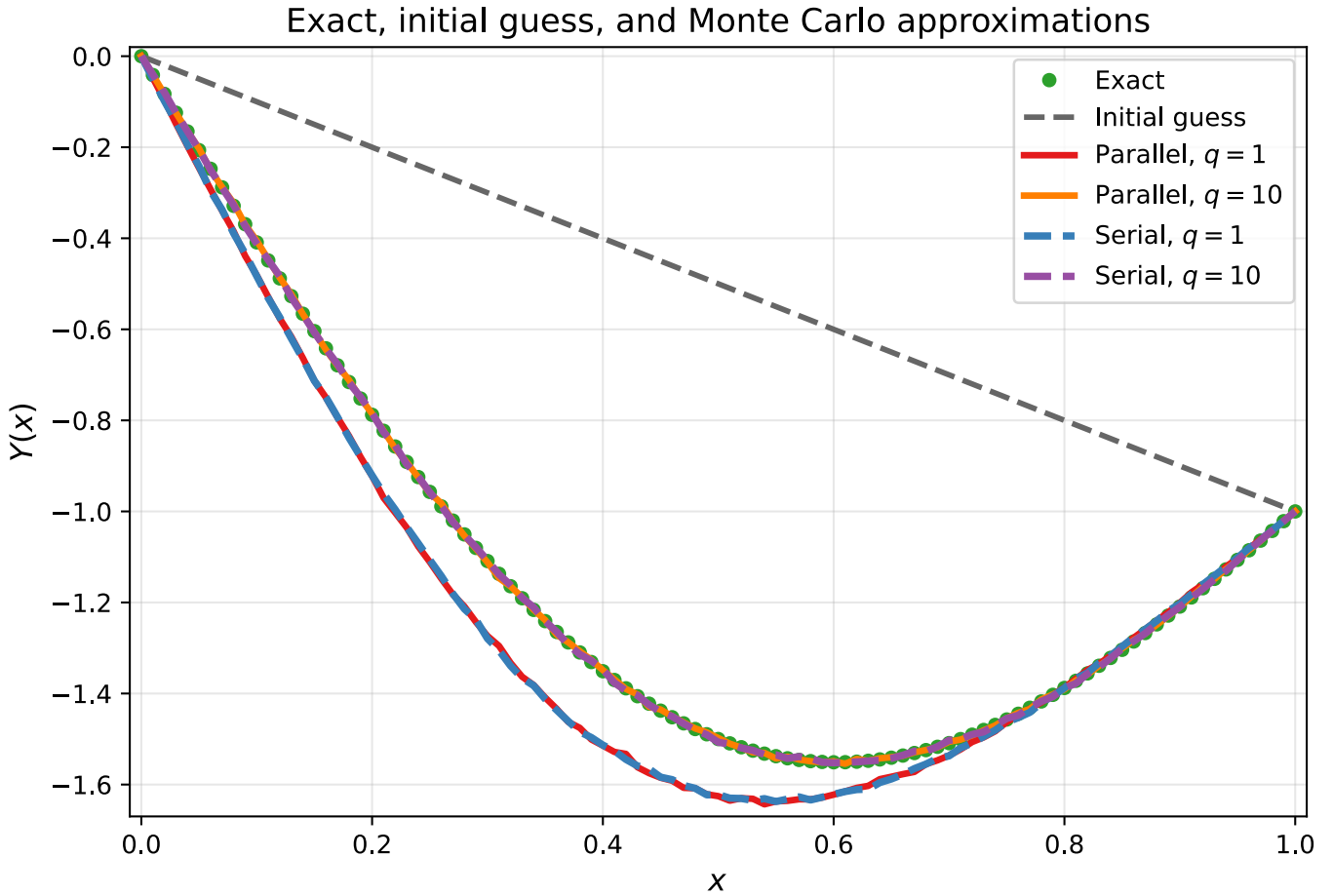


Figura 1: Resultados de este trabajo.

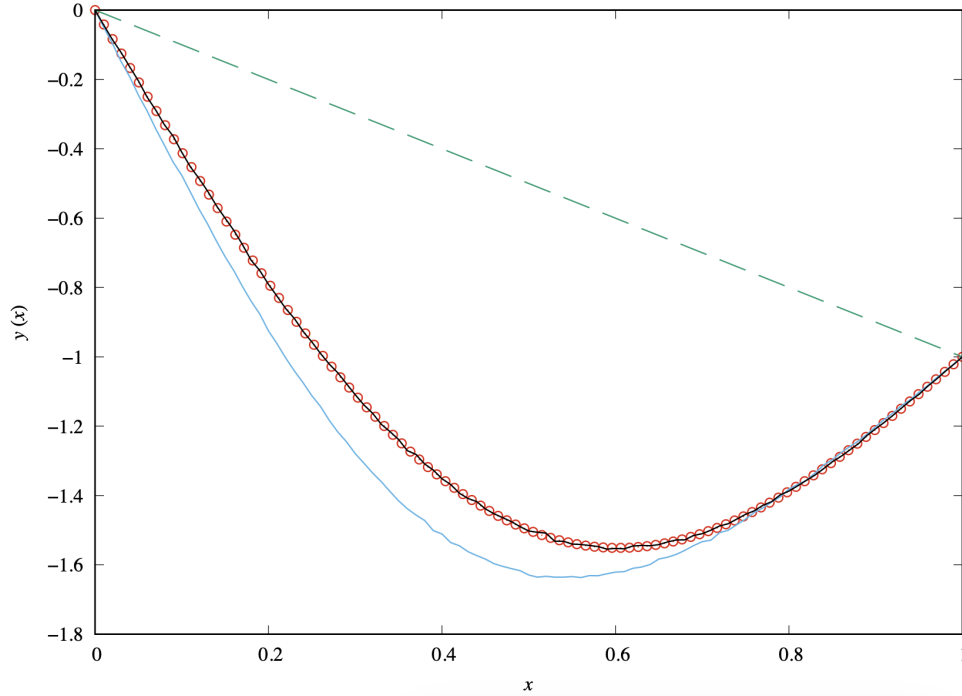


Figura 2: Resultado tomado de [1].

Conclusiones/ Discusión

El método presentado en [1] es destacable porque permite resolver cualquier problema de Dirichlet, como se ha indicado. Su desventaja es que se requiere un gran número de caminantes o iteraciones para aumentar la precisión, lo cual incrementa el tiempo de cómputo debido al uso de condiciones absorbentes.

La comparación con la implementación serial demuestra que la paralelización no altera la exactitud de los resultados y ofrece una mejora en el rendimiento computacional puesto que este método es *embarrassingly parallel*. La principal limitante de esta paralelización radica en que se ejecuta en CPU usando threads; usar una GPU debería ofrecer un rendimiento mucho mayor, pues los CPU's no fueron diseñados para una paralelización masiva [2].

La paralelización de este algoritmo se ha realizado de forma que aprovecha las herramientas de `numba` sin caer en sus limitaciones: `numba` no puede paralelizar ni una función que llame a otra también paralelizada ni dos ciclos anidados.

Referencias

- [1] Martín Chávez, Enrique González y Guillermo Iván Guerrero-García, *An iterative Monte Carlo method to solve nonlinear second-order differential equations*, Disponible en: <https://arxiv.org/pdf/2408.00742>
- [2] Peter S. Pacheco, *An Introduction to Parallel Programming*, Elsevier, 2015.